

示例项目

整体的使用，参考示例项目：`simple-example-app`

初始账号-admin

管理员初始帐号/密码：`admin/admin`

详情

springboot

springboot 的版本：2.7.15

封装返回值

如果需要对接口的返回值进行自动封装统一结构，需要实现接口 `WrapperResponseScanPackages` 并注册为bean 对象

错误码枚举

自定义错误码枚举 `ErrorCodeEnums`(类名自定义) 并实现接口 `IErrroCode`

同时需要创建文件 `classpath:/META-INF/services/com.base.web.error.IErrroCode` 并将该枚举类的完全限定类名写在里面

这里的作用是检查所定义的错误码是否有重复的code 值，如果未配置则无法检查错误码重复的问题

约定 code 在 2,000,000 以内的数字留给sdk，业务相关的从 2,000,000 开始使用

国际化配置

sdk 中的国际化配置文件目录为 message 如果业务的目录为其他，则需要将两个目录都配置上，如下：

```
1  spring:
2    messages:
3      # 要使用MessageSource 我们应该要提供 一个对应 的配置文件 。
4      # 当前SDK 中，使用的目录是: "messages"
5      basename: "messages,i18n/messages"
6      fallback-to-system-locale: false
```

如果配置文件中找不到对应的key，则直接使用代码中枚举对应的msg。

全局异常处理

全局异常拦截处理已经添加，业务异常直接使用 `com.base.web.exception.ExceptionUtil`，示例如下：

```
1 | throw ExceptionUtil.business(BaseWebErrorCodeEnums.SERVICE_ERROR)
```

数据库

1. 支持多数据源以及flywaydb

由于兼容性问题，需要控制flyway-core 的版本号

```
1 | <flyway.version>7.15.0</flyway.version>
2 | <dependency>
3 |     <groupId>org.flywaydb</groupId>
4 |     <artifactId>flyway-core</artifactId>
5 |     <version>${flyway.version}</version>
6 | </dependency>
```

2. sdk 所需要的用户相关的SQL 存放在目录： `classpath:/sqls/mysql/sys/base`

同时sdk 中的flyway 版本文件名采用格式： `v00_00_00_xxx_xxxxx.sql` 业务相关的版本文件名需要与其区分，**不要冲突了**

3. 如果启用flyway 则需要将 sdk 相关的SQL 目录配置上

4. sdk 所使用的数据源为默认数据源，所以用户相关的数据源需要处理为默认数据源，sql 目录配置也应该配置在默认数据源上面

5. 多数据源配置示例

首先，必须先禁用掉 Flyway 的自动配置

```
1 | spring:
2 |     flyway:
3 |         # 系统实现的 FlywayAutoConfiguration 需要禁用掉
4 |         enabled: false
```

然后添加相关数据源的配置

```
1 | spring:
2 |     datasource:
3 |         dynamic:
4 |             enabled: true
5 |             strict: true
6 |             # 这里的值，要从下面的值中选择一下。
7 |             # 这里的 "master" 就是 base.datasource.hikari 的下一级中选择的一个。
8 |             primary: "master"
9 |
10 | base:
11 |     datasource:
12 |         hikari:
13 |             # 这里的master 就是上面配置的值(spring.datasource.dynamic.primary)
14 |             master:
```

```

15         jdbc-url: jdbc:mysql://192.168.8.143:3306/example_master?
            useUnicode=true&characterEncoding=utf8&zeroDateTimeBehavior=convertT
            oNull&useSSL=false&rewriteBatchedStatements=true&serverTimezone=Asia
            /Shanghai
16         username: example_master
17         password: example_master
18         flyway:
19             enabled: true
20             locations:
21                 - "sqls/mysql/sys/base"
22                 - "sqls/mysql/master"
23         slave:
24         jdbc-url: jdbc:mysql://192.168.8.143:3306/example_slave?
            useUnicode=true&characterEncoding=utf8&zeroDateTimeBehavior=convertT
            oNull&useSSL=false&rewriteBatchedStatements=true&serverTimezone=Asia
            /Shanghai
25         username: example_slave
26         password: example_slave
27         flyway:
28             enabled: true
29             locations:
30                 - "sqls/mysql/slave"

```

用户认证

引入了spring-security 但是没有使用它的用户认证体系，需要将其相关的自动配置排除掉。

添加如下配置项：

```

1  spring:
2      autoconfigure:
3          exclude:
4              # 禁用spring-security 的用户相关的功能
5              # 其主要作用是为 Spring Boot 应用提供默认的用户认证机制
6              -
            org.springframework.boot.autoconfigure.security.servlet.UserDetailsServiceAut
            oConfiguration

```

日志

日志，使用 logback，直接在配置文件中配置日志目录以及文件名即可。

默认日志文件： `/tmp/unknown.log`

如下配置，日志文件将会生成： `/tmp/app/sys-info.log`

```

1  logging:
2      file:
3          path: "/tmp/app"
4          name: "sys-info"

```

swagger 配置

```
1 knife4j:
2   enable: true
3
4   # swagger 页面访问: http://localhost:${server.servlet.context-path}/doc.html
5   springdoc:
6     group-configs:
7       # 分组信息也可以在这里配置, 在代码的配置类里面配置也是可以的。
8       - group: 'ruoyi-system-user'
9         paths-to-match:
10          - "/system/user/**"
11         packages-to-scan:
12          - "com.web.ruoyi.controller"
```

审计日志

审计日志 (页面操作日志记录, 表: operation_record)

自定义审核枚举, 实现接口 `IAudit`, 参考 `SysWebAuditEnums`

用法: 在需要日志的接口上面添加注解 `@AuditOperation`

具体参考: `SysUserController.edit`

```
1   @AuditOperation("@audit.auditRecord(" +
2
3   "T(com.web.sys.constants.enums.SysWebAuditEnums).SYSTEM_USER_EDIT, " +
4   "    #spelReturnValue, #request, #loginUser, " +
5   "    #user)")
6   @PostMapping()
7   public AjaxResult edit(
8       @SuppressWarnings("unused") HttpServletRequest request,
9       @Parameter(hidden = true) @CurrLoginUser LoginUser loginUser,
10      @Validated @RequestBody SysUser user)
11  {
12      return toAjax(editUser(loginUser, user));
13  }
```

忽略认证

1. 默认情况下, 有几种接口是不需要认证的, 参考:

`AbstractAuthenticationInterceptor#ignoreAuthPathPatterns`

2. 对于controller 方法, 使用注解 `@Permit(required = false)` 即可。

3. 对于其他路径, 需要实现接口 `IgnoreAuthPathPatternProvider` 并注册为bean 对象。

用户token

用户token 密钥配置

如果未配置, 有一个默认的字符串

```
1 sys:
2   web:
3     user:
4       token-secret-key: "r7X9cLp5Z2v8kbJ3n6F7d4H1m9s0Q8w"
```

excel 导入导出

- 导入 (`RuoyiExcelUtil`)
- 导出 (`ExcelExport`)

菜单历史记录

首个版本见: `sys-menu-v0.0.0.txt`

```
1  -- 菜单表数据，用于每次版本升级对菜单调整的对比。
2  SELECT
3      menu_id, menu_name, parent_id, order_num, path, component, query,
4      route_name, is_frame, is_cache, menu_type, visible, status, perms,
5      icon, remark, menu_key
6  FROM sys_menu
7  ORDER BY
8      menu_id, menu_name, parent_id, order_num, path, component, query,
9      route_name, is_frame, is_cache, menu_type, visible, status, perms,
10     icon, remark, menu_key;
```